

Documentación técnica del pipeline predictivo de matrícula universitaria chilena

Documento de arquitectura, metodología y defensa técnica

Autor: Diego Evans Pérez Paz

Versión del pipeline: v3 (dataset 75.622×51)

Fecha: Junio de 2026

Propósito: Documentación técnica exhaustiva orientada a la auditoría y la defensa metodológica del modelo predictivo de matrícula universitaria.

Cómo leer este documento

Este documento está organizado en seis partes y tres apéndices. No es un tutorial ni un resumen ejecutivo: es la **memoria técnica completa** del pipeline, diseñada para que un ingeniero de datos o científico de datos pueda (i) comprender exactamente qué se hizo y por qué, (ii) reproducir cada paso, y (iii) auditar críticamente cada decisión de diseño.

- **Parte I (Fundamentos)** establece el problema, la nomenclatura y la formalización matemática.
 - **Parte II (Datos y trazabilidad)** documenta las fuentes y —de forma central— la auditoría completa de cómo el número de matrículas se conserva o reduce en cada etapa del pipeline, partiendo del SIES puro.
 - **Parte III (Consolidación)** detalla el merge multifuente, la normalización de llaves y la imputación.
 - **Parte IV (Modelado)** justifica cada decisión: validación temporal, tratamiento del shock pandémico, target encoding, selección de modelos y el experimento de clustering descartado.
 - **Parte V (Resultados)** interpreta las métricas, explicando qué dicen y qué no dicen.
 - **Parte VI (Blindaje)** anticipa ~20 objeciones técnicas adversariales con respuesta empírica.
-

PARTE I — FUNDAMENTOS

1. Propósito y alcance

1.1 Qué resuelve este pipeline

El pipeline construye y valida un modelo predictivo de **matrícula universitaria chilena** a granularidad fina. La unidad de observación es la combinación:

```
institución × carrera × región × jornada × género × condición × año
```

donde **condición** distingue entre alumnos **nuevos** (primer año) y **antiguos** (cohortes superiores). El objetivo es predecir cuántos estudiantes habrá en cada una de estas combinaciones para un año dado, generando tanto predicciones retrospectivas validadas (2019-2025) como un pronóstico a futuro (2026).

1.2 Alcance y delimitación explícita

- **Universo cubierto:** exclusivamente universidades (62 instituciones). Se excluyen Institutos Profesionales (IP) y Centros de Formación Técnica (CFT). Esta delimitación es deliberada y se discute en la Parte VI.
- **Período:** 2018-2025 para entrenamiento y validación; 2026 como horizonte de pronóstico. El año 2017 del SIES se utiliza únicamente como base para el cálculo del lag de 2018.
- **Naturaleza:** modelo **predictivo**, no causal. Las contribuciones SHAP identifican asociaciones estadísticas, no relaciones causa-efecto.

1.3 Los tres notebooks del pipeline

Notebook	Rol	Output principal
auditoria_y_merge_consolidado_v2_FINAL.ipynb	Construcción del dataset: carga, normalización, merge, imputación, feature engineering	dataset_modelado_f51)
modelado_predictivo_admision_v3.ipynb	Entrenamiento, validación temporal, tuning, evaluación, forecast	predicciones_model + modelo .joblib
analisis_paper_resultados.ipynb	Análisis post-hoc: métricas, figuras, tablas	12 figuras + 9 tablas

Notebook	Rol	Output principal
	para publicación	

2. Glosario técnico y convenciones

2.1 Términos clave

Término	Definición operacional en este pipeline
Target	Tasa de crecimiento interanual de matrícula, transformada con arcsinh . Ver §3.2.
Lag (<code>matriculas_lag1</code>)	Matrícula de la misma combinación el año anterior ($t - 1$). Variable predictora central.
OOF (out-of-fold)	Predicción para un año generada por un modelo que NO vio ese año en entrenamiento.
Granularidad fina	El nivel más desagregado (la unidad de observación completa).
Holdout	Año 2025, reservado y nunca usado en selección de modelo ni tuning.
Expanding window	Esquema de CV donde la ventana de entrenamiento crece año a año. Ver §10.
Target encoding (TE)	Codificación de categóricas de alta cardinalidad por la media del target. Ver §12.
Fold-aware	Propiedad de que el TE se recalcula dentro de cada fold, sin ver datos de validación.
WAPE	Weighted Absolute Percentage Error: $\$100 \cdot \sum$
Skill score	Mejora relativa del RMSE ² respecto a un baseline naïve. Ver §15.3.
Shock de nivel	Año donde la media del target se desvía significativamente (Z-score). Ver §11.

2.2 Convenciones de nomenclatura de columnas

- **Categóricas originales** (para TE fold-aware): `REGIÓN` , `NOMBRE INSTITUCIÓN` , `NOMBRE CARRERA` , `area_conocimiento` .
- **Columnas `te_*`** : target encoding precalculado. **Existen en el dataset pero NO se usan en el modelo final** — sirven exclusivamente como baseline en el experimento de leakage (§12). Su inclusión en el modelo constituiría leakage.

- **Columnas** `es_shock_*` : dummies binarias del tratamiento pandémico.
- **Prefijo** `clasificacion3_` : dummies de estado de acreditación.
- **Variables con prefijo de jornada/condición** (`JORNADA_*` , `CONDICION_*`): one-hot encoding.

2.3 Particularidades del entorno chileno

Todo el pipeline asume convenciones de locale español de Chile:

- **Separador CSV:** `;` (punto y coma), porque la coma es separador decimal.
- **Separador decimal:** `,` (coma).
- **Codificación:** `utf-8-sig` (UTF-8 con BOM), requerida tanto en lectura como en escritura para preservar caracteres como `Ñ` , `ó` , `í` .

⚠ **Nota de ingeniería crítica:** exportar un CSV sin `sep=';', decimal=','` en un entorno Windows con locale español provoca que los valores numéricos se interpreten inflados $\sim 1000\times$ al reabrirlos en Excel (porque el `.` se lee como separador de miles). Esta es una fuente de error silencioso que el pipeline previene explícitamente en cada export.

3. Formalización del problema

3.1 Por qué predecir tasa de crecimiento y no nivel absoluto

Una decisión de diseño fundamental: el modelo **no predice directamente el número de matrículas**, sino la **tasa de crecimiento interanual**. La matrícula absoluta se reconstruye después a partir de la tasa predicha y el lag conocido.

La razón es estadística. El número de matrículas tiene una distribución extremadamente heterogénea: una carrera de medicina en una universidad grande puede tener 800 alumnos, mientras que un programa nicho tiene 12. Modelar el nivel absoluto directamente haría que el modelo se concentrara en acertar los grandes (que dominan el error cuadrático) e ignorara los pequeños. Al modelar la **tasa de crecimiento**, todas las combinaciones quedan en una escala comparable, y el lag absorbe la información de nivel.

3.2 Definición matemática del target

La tasa de crecimiento se define como:

$$\text{tasa}_t = \frac{\text{matrículas}_t - \text{matrículas}_{t-1}}{\text{matrículas}_{t-1}}$$

Esta tasa cruda tiene colas pesadas y asimetría. Para mejorar las propiedades de entrenamiento, se aplica la transformación **arcsinh** (seno hiperbólico inverso):

$$\text{target}_t = \text{arcsinh}(\text{tasa}_t) = \ln \left(\text{tasa}_t + \sqrt{\text{tasa}_t^2 + 1} \right)$$

Por qué arcsinh y no logaritmo: el logaritmo no admite valores ≤ 0 , y las tasas de crecimiento son frecuentemente negativas (carreras que decrecen). El arcsinh es simétrico, está definido en todo $\mathbb{R}$, preserva el signo, y se comporta como la identidad cerca de cero y como el logaritmo en las colas. Es la transformación natural para variables con signo y colas pesadas.

3.3 Reconstrucción de matrículas absolutas

Para evaluar el modelo en la escala que importa (número de estudiantes), se invierte la transformación:

$$\begin{aligned} \hat{\text{tasa}}_t &= \sinh(\text{target}_t) \\ \widehat{\text{matrículas}}_t &= \text{matrículas}_{t-1} \times (1 + \hat{\text{tasa}}_t) \end{aligned}$$

Esta distinción entre **escala de optimización** (arcsinh de la tasa) y **escala de evaluación** (matrículas absolutas) es central para interpretar correctamente las métricas, y se discute en profundidad en §15.1.

3.4 Winsorización previa

Antes de la transformación arcsinh, la tasa cruda se winsoriza a los percentiles 1 y 99:

```
p01 = df_model['tasa_crecimiento'].quantile(0.01)
p99 = df_model['tasa_crecimiento'].quantile(0.99)
df_model['tasa_crecimiento'] = df_model['tasa_crecimiento'].clip(lower=p01,
upper=p99)
df_model['tasa_crecimiento'] = np.arcsinh(df_model['tasa_crecimiento'])
```

Esto limita el efecto de outliers extremos (carreras que duplican o colapsan su matrícula por razones administrativas, no por dinámicas de demanda) sin eliminarlos del dataset.

PARTE II — DATOS Y TRAZABILIDAD

4. Inventario de fuentes

4.1 Las diez fuentes integradas

#	Fuente	Organismo	Granularidad original	Rol en el modelo
1	Matrículas SIES	MINEDUC	institución × carrera × jornada × género × condición × año	Tabla base + target
2	BaseINDICES (aranceles, puntajes, vacantes)	CNED	institución × carrera × año	Features de precio y selectividad
3	Cuerpo Docente	CNED	institución × año	Features de claustro (ratios)
4	Inmuebles	CNED	institución × sede × año	Features de infraestructura
5	PIB regional	Banco Central	región × año	Feature macroeconómico
6	Tasa de desocupación	INE (SIMEL)	región × sexo × año	Feature macroeconómico
7	Empleabilidad e Ingresos	mifuturo.cl	institución × carrera × año	Features de retención/empleabilidad
8	Diccionario sede- región	CNED	sede → código región INE	Llave de normalización
9	Diccionario de instituciones	SIES	catálogo de 222 instituciones	Llave canónica institucional
10	Diccionario de carreras	SIES	catálogo de 15.940 carreras	Llave canónica de carreras

4.2 Particularidades por fuente (lo que un ingeniero debe saber)

SIES (fuente base): usa su propio sistema de códigos institucionales (rango 1-60 para universidades). Crucialmente, estos códigos coinciden 60/60 con el diccionario canónico de instituciones, por lo que SIES no requiere normalización por nombre. La variable `area_conocimiento` proviene de aquí con 100% de cobertura.

Fuentes CNED (docente, inmuebles, aranceles) y mifuturo (retención): usan sistemas de códigos institucionales **distintos** del SIES (rangos 1001+, 3001+). Esto obliga a mapearlas al código SIES vía **nombre normalizado**, lo cual introduce la necesidad de un normalizador agresivo y de overrides manuales (ver §7).

PIB regional y desempleo: granularidad regional, no institucional. Se unen por `año + código de región`. El desafío técnico aquí es la normalización de los nombres de región, que vienen

con codificaciones inconsistentes entre fuentes (ver §7.1).

Codificación de archivos: todas las fuentes chilenas usan `utf-8-sig` con separadores `;` y `,`. Varias contienen caracteres `\r\n` espurios en los nombres de institución que deben limpiarse antes de normalizar.

5. ★ Auditoría de trazabilidad de matrículas

Esta es la sección central para la auditoría. Documenta exactamente cuántas filas y cuántas matrículas existen en cada etapa del pipeline, partiendo del SIES crudo, y justifica cada reducción. **Cualquier auditor puede reproducir esta tabla descargando el SIES público y siguiendo los stages.**

5.1 Tabla maestra de stages

El notebook de consolidado calcula esta tabla automáticamente en su celda final. Los números son los siguientes:

Total de FILAS por año en cada stage

Año	S1: SIES crudo	S2: filtrado universidades	S3: post- merge	S4: con target	S5: modelado final
2018	34.653	15.167	15.047	14.161	9.655
2019	34.090	14.738	14.652	14.000	9.715
2020	33.466	14.317	14.232	13.720	9.532
2021	34.527	14.312	14.212	13.586	9.403
2022	34.950	14.330	14.227	13.838	9.423
2023	35.643	14.099	14.021	13.635	9.309
2024	34.984	13.679	13.602	13.290	9.406
2025	34.565	13.470	13.393	12.920	9.179
Total (2018- 2025)	276.878	114.112	113.386	109.150	75.622

Nota: el S1 total (630.554 filas) incluye los años 2007-2017 que no entran al modelo. La columna de total 2018-2025 del S1 es 276.878.

Total de MATRÍCULAS por año en cada stage

Año	S1: SIES crudo	S2: filtrado	S3: post- merge	S4: con target	S5: modelado final
2018	1.188.045	678.213	677.018	659.175	642.735
2019	1.194.459	677.084	676.385	665.836	652.325
2020	1.151.834	660.109	659.323	651.618	635.876
2021	1.204.376	691.375	690.249	677.632	662.345
2022	1.214.011	685.435	684.674	676.343	658.118
2023	1.249.419	693.662	692.957	687.730	669.747
2024	1.277.611	706.040	705.248	699.366	691.940
2025	1.327.344	731.981	730.943	719.155	706.843

Porcentaje de retención de matrículas entre stages

Año	S2/S1 (% universitario)	S3/S2 (merge)	S4/S3 (target)	S5/S4 (lag≥10 + dedup)
2018	57.09%	99.82%	97.36%	97.51%
2019	56.69%	99.90%	98.44%	97.97%
2020	57.31%	99.88%	98.83%	97.58%
2021	57.41%	99.84%	98.17%	97.74%
2022	56.46%	99.89%	98.78%	97.31%
2023	55.52%	99.90%	99.25%	97.39%
2024	55.26%	99.89%	99.17%	98.94%
2025	55.15%	99.86%	98.39%	98.29%

5.2 Justificación de cada caída de registros

La interpretación de la tabla, stage por stage:

S1 → S2 (de 276.878 a 114.112 filas; retención de matrículas 55-57%). Esta es la mayor reducción y es **completamente deliberada y esperada**. Corresponde a dos filtros:

- 1. **Filtro temporal:** se descartan los años 2007-2017 (el modelo opera sobre 2018-2025).
- 2. **Filtro de universo:** se conservan solo las tres categorías universitarias (Universidades CRUCH , Universidades Privadas , Universidades (* Carrera en Convenio)),

descartando IP y CFT.

El hecho de que la retención de matrículas sea ~57% y descienda a ~55% no es una pérdida de datos: **es la cuantificación directa de la participación universitaria en el sistema de educación superior chileno**. Las matrículas "perdidas" aquí son los estudiantes de IP/CFT, que están fuera del alcance del modelo por diseño. El descenso de 57.09% (2018) a 55.15% (2025) es precisamente el fenómeno estructural de pérdida de participación universitaria documentado en el paper.

S2 → S3 (de 114.112 a 113.386 filas; retención de matrículas ~99.87%). El merge con las seis fuentes secundarias es un **left join estricto desde SIES**. El código incluye un assert que garantiza que el número de filas no aumenta (no hay inflado por relaciones uno-a-muchos mal resueltas). La pérdida marginal de ~726 filas (0.6%) corresponde a filas con valores de llave de región nulos que no sobreviven la reconstrucción del lag. La conservación del 99.87% de matrículas confirma que el merge es esencialmente sin pérdida.

S3 → S4 (de 113.386 a 109.150 filas; retención de matrículas ~98.5%). Aquí se descartan las filas para las que **no es posible calcular el target** (tasa de crecimiento). Esto ocurre cuando una combinación aparece por primera vez en el dataset y no tiene un año previo contra el cual calcular crecimiento. Es una limitación matemática inherente: no se puede calcular crecimiento sin un punto de referencia. La cobertura del target es del 96.3% a nivel de filas.

S4 → S5 (de 109.150 a 75.622 filas; retención de matrículas ~97-99%). Esta caída tiene dos componentes auditables:

1. **Filtro `matriculas_lag1 ≥ 10`**: elimina combinaciones con menos de 10 estudiantes el año previo. La razón es numérica: una carrera que pasa de 2 a 6 estudiantes tiene +200% de crecimiento, que es ruido puro y no señal de demanda. Estas combinaciones de cola larga inflarían artificialmente la varianza del target.
2. **Eliminación de duplicados exactos**: 13.238 filas duplicadas tras las transformaciones (combinaciones que, una vez aplicado OHE y demás, resultan idénticas).

El punto crítico para la auditoría: aunque S4→S5 elimina el 31% de las *filas*, conserva el **97-99% de las matrículas** cada año. Esto demuestra que las filas eliminadas son combinaciones minúsculas (cola larga de programas con poquísimos estudiantes), no programas masivos. La señal agregada se preserva casi íntegramente.

5.3 Cómo auditar desde el SIES puro (procedimiento reproducible)

Un auditor externo puede verificar la cobertura del modelo con este procedimiento, usando exclusivamente el archivo SIES público:

```

import pandas as pd
import numpy as np

# 1. Cargar SIES crudo (archivo público de mifuturo.cl)
sies = pd.read_csv('SIES_MATRICULA.csv', sep=';', decimal=',', encoding='utf-8-sig')

# 2. Replicar el filtro de universo
FILTRO_UNIVERSIDADES = [
    'Universidades CRUCH',
    'Universidades Privadas',
    'Universidades (* Carrera en Convenio)',
]
sies_uni = sies[sies['CLASIFICACIÓN INSTITUCIÓN NIVEL 2'].isin(FILTRO_UNIVERSIDADES)]
sies_uni = sies_uni[sies_uni['AÑO_AUX'].between(2018, 2025)]

# 3. Verificar el total de matrículas universitarias por año
control = sies_uni.groupby('AÑO_AUX')['TOTAL_MATRICULAS'].sum()
print(control)
# Debe coincidir con la columna S2 de la tabla maestra:
# 2018: 678.213 | 2019: 677.084 | ... | 2025: 731.981

```

Si estos números coinciden con la columna S2 de la tabla maestra (§5.1), el auditor ha verificado que el punto de partida del pipeline es correcto y que no hay manipulación en la base. A partir de ahí, cada stage subsiguiente está documentado y es reproducible ejecutando el notebook de consolidado.

5.4 Reconciliación final: cobertura del 96.3%

La afirmación de que el modelo cubre el **96.3% de las matrículas universitarias** se calcula así: del SIES filtrado a universidades (S2), las filas con target calculable (S4) representan el 96.3% de las filas. Las matrículas en el dataset final (S5) representan, año a año, entre el 94.8% y el 96.9% del total universitario S2:

Año	Matrículas S5 / Matrículas S2
2018	642.735 / 678.213 = 94.8%
2019	652.325 / 677.084 = 96.3%
2020	635.876 / 660.109 = 96.3%
2021	662.345 / 691.375 = 95.8%
2022	658.118 / 685.435 = 96.0%

Año	Matrículas S5 / Matrículas S2
2023	669.747 / 693.662 = 96.6%
2024	691.940 / 706.040 = 98.0%
2025	706.843 / 731.981 = 96.6%

Esta cobertura del ~96% es **alta y honesta**: el modelo no descarta selectivamente programas para mejorar sus métricas, sino que pierde únicamente la cola larga de combinaciones minúsculas y las primeras apariciones sin lag.

(Continúa en la Parte III — Consolidación)

PARTE III — PIPELINE DE CONSOLIDACIÓN

6. Arquitectura del merge

6.1 Por qué un diseño SIES-céntrico

El consolidado se construye con SIES como **tabla ancla** y todas las demás fuentes unidas mediante **left join** desde ella. Esta decisión tiene una justificación precisa: SIES es la única fuente que contiene la variable objetivo (matrícula) a la granularidad requerida. Las demás fuentes aportan *features* contextuales (precio, claustro, infraestructura, macroeconomía), pero ninguna define la unidad de observación.

Un left join desde SIES garantiza que:

1. **Ninguna fila de matrícula se pierde** por ausencia en una fuente secundaria (un programa sin datos de aranceles sigue existiendo, con arancel a imputar).
2. **No se introducen filas espurias** desde las fuentes secundarias (no aparecen combinaciones que no existen en la matrícula real).

6.2 Garantía de no-inflado

El riesgo técnico de cualquier merge es el **inflado por relaciones uno-a-muchos**: si una fuente secundaria tiene múltiples filas por la misma llave, el join multiplicaría las filas de SIES. El pipeline previene esto con un assert explícito tras cada merge:

```
consolidado = sies_filtrada.copy()
n_inicial = len(consolidado)
```

```
# ... seis merges secuenciales (PIB, desempleo, docente, inmuebles, retención,
aranceles) ...

assert len(consolidado) == n_inicial, f'INFLADO: {len(consolidado):,} vs
{n_inicial:,}'
print(f'✓ Sin inflado de filas: {len(consolidado):,}')
```

Para que este assert pase, las fuentes secundarias se **pre-agregan** a la granularidad exacta de su llave antes del merge. Por ejemplo, aranceles se agrega a (año, región, institución, carrera_norm) con funciones de agregación explícitas, garantizando unicidad de la llave.

6.3 Las llaves de merge por fuente

Fuente	Llave de merge	Granularidad
PIB regional	año + cod_region	Regional
Desempleo	año + cod_region	Regional
Cuerpo docente	año + cod_region + cod_institucion	Institucional
Inmuebles	año + cod_region + cod_institucion	Institucional
Retención	año + cod_institucion + carrera_norm	Programa
Aranceles	año + cod_region + cod_institucion + carrera_norm	Programa-región

7. Normalización de llaves

7.1 Normalización de regiones

Las regiones llegan con codificaciones inconsistentes entre fuentes. El problema más insidioso es el **doble encoding de caracteres especiales**: la región de Ñuble aparece como Ã'uble en algunas fuentes (UTF-8 mal interpretado como Latin-1 y re-codificado). El pipeline incluye una reparación explícita, con una consideración de orden crítica:

```
# FIX: corregir mapping de Ñuble y re-aplicar el mapeo
# El doble-encoding debe repararse DESPUÉS de asegurar que cod_region es
string
```

⚠ **Lección técnica documentada:** la reparación del doble-encoding debe ejecutarse *después* de coercionar el tipo de la columna a string. Si se ejecuta antes de que la coerción de tipo esté completa, el fix string-based no encuentra el patrón y Ñuble queda mal mapeada silenciosamente. Este es un bug que efectivamente ocurrió y se corrigió.

7.2 ★ Normalización de instituciones y los cinco overrides

Las cuatro fuentes CNED/mifuturo usan códigos institucionales distintos del SIES. Se mapean al código canónico vía nombre normalizado mediante un **normalizador agresivo**:

```
def normalizar_institucion(s):
    if pd.isna(s):
        return s
    s = normalizar(s) # mayúsculas, sin tildes, sin
    puntuación
    s = re.sub(r'\bC F T\b', 'CFT', s)
    s = re.sub(r'\bI P\b', 'IP', s)
    # FIX v2: "U" sola → "UNIVERSIDAD" en CUALQUIER posición (no solo al
    inicio)
    s = re.sub(r'\bU\s', 'UNIVERSIDAD ', s)
    for pattern, replacement in ALIASES_INSTITUCION.items():
        s = re.sub(pattern, replacement, s)
    s = ' '.join(s.split())
    return s
```

A pesar de la normalización agresiva, **cinco universidades no matchean** por variantes de naming irreductibles (apóstrofes tipográficos, guiones, siglas comerciales). Para estas se definen overrides manuales explícitos:

```
OVERRIDES_INSTITUCION = {
    'UNIVERSIDAD BERNARDO O'HIGGINS': 50,
    'UNIVERSIDAD DE O'HIGGINS': 895,
    'UNIVERSIDAD DE ARTES, CIENCIAS Y COMUNICACION UNIACC': 26,
    'UNIVERSIDAD CHILENO-BRITANICA DE CULTURA': 651,
    'UNIVERSIDAD IBEROAMERICANA DE CIENCIAS Y TECNOLOGIA': 25,
}
```

Por qué overrides manuales y no más reglas de normalización: agregar reglas genéricas para resolver estos cinco casos arriesgaría producir colisiones en otras instituciones (por ejemplo, una regla que normalice "Bernardo O'Higgins" podría afectar a "De O'Higgins"). Los overrides explícitos son auditables, no tienen efectos colaterales, y son la práctica estándar para casos irreductibles. El número de overrides (5 sobre 62) es bajo y está documentado.

7.3 Resultado de cobertura institucional

Tras la normalización + overrides, la cobertura SIES-céntrica es:

Fuente	Instituciones cubiertas
DOCENTE	60 / 60 (100%)
INMUEBLES	60 / 60 (100%)
ARANCELES	60 / 60 (100%)
RETENCIÓN	58 / 60 (96.7%)

Las dos universidades sin datos de retención simplemente reciben imputación en esa variable; no se pierden del dataset.

7.4 Cobertura a nivel de combinaciones de llave

La cobertura institucional (qué instituciones matchean) es distinta de la cobertura de combinaciones (qué filas específicas tienen dato). Esta última, medida SIES-céntricamente, es:

Fuente	Combinaciones SIES	Match	% cobertura
PIB	128	120	93.75%
DESEMPLEO	128	128	100.0%
DOCENTE	1.090	842	77.25%
INMUEBLES	1.090	848	77.80%
RETENCIÓN	20.462	4.844	23.67%
ARANCELES	27.558	17.327	62.87%

Estas coberturas parciales son la razón por la que se necesita un sistema de imputación robusto (§8). La baja cobertura de retención (23.67%) refleja que mifuturo.cl no reporta retención para todas las combinaciones programa-año; estas se imputan jerárquicamente.

8. Sistema de imputación jerárquica

8.1 Filosofía: imputar de lo específico a lo general

Las variables de aranceles, retención, docentes e inmuebles tienen cobertura parcial. En lugar de imputar con un único valor global (que destruiría la señal), se diseñó un sistema de **cinco**

capas jerárquicas. Cada capa imputa solo las celdas que la capa anterior dejó vacías, yendo de la agrupación más específica (que preserva más señal) a la más general.

8.2 Las cinco capas

Capa	Variables	Lógica de agrupación	% del trabajo de imputación
1 — Temporal	Docente, Inmuebles	Interpolación lineal por (institución, región) sobre el eje año, con ffill/bfill	4.2%
2 — Jerárquica	Aranceles, Retención	Cascada: (institución, carrera) → (institución, área, año) → (área, año)	69.8%
2.5 — Región+carrera	Aranceles, Retención	Cross-institución: (región, carrera, año) → (carrera, año) → (carrera)	0.0%
2.7 — Institución	Aranceles, Retención	(institución, año) → (institución)	0.0%
3 — Terminal	Todas	Mediana global de la variable	26.0%

8.3 Por qué la Capa 2 absorbe el 69.8% (y por qué eso es bueno)

La Capa 2 utiliza la jerarquía natural del conocimiento académico: si no hay arancel para una carrera específica, se usa el arancel mediano de esa área de conocimiento en esa institución y año. Esta es una imputación **informativa**: un programa de ingeniería sin dato hereda el comportamiento típico de ingeniería en su institución, no un valor global sin sentido.

El hecho de que el 69.8% de la imputación se resuelva en esta capa de alta calidad (y solo el 26% caiga a mediana global) es una mejora estructural sustantiva. Es posible gracias a la **cobertura del 100% de area_conocimiento** proveniente de SIES: sin esa variable, la jerarquía por área no funcionaría y el grueso de la imputación caería a la capa terminal.

Nota sobre las capas 2.5 y 2.7 con 0.0%: estas capas existen en el código como red de seguridad, pero en la práctica la Capa 2 ya resuelve todo lo que ellas habrían cubierto. Se mantienen por robustez ante futuros datasets con distinta estructura de faltantes.

8.4 Trazabilidad de la imputación

Cada fila lleva una flag binaria `fue_imputado` que indica si al menos una de sus variables fue imputada. El 73.4% de las filas tiene alguna imputación. Esta flag se incluye como feature en el modelo, permitiéndole **conocer y ajustar por** el hecho de que una observación tiene datos

imputados. No se oculta la imputación: se hace explícita y se le da al modelo la capacidad de ponderarla.

9. Feature engineering

9.1 Las diez features derivadas

Sobre las variables crudas integradas, se construyen diez features de ingeniería, cuatro de las cuales usan lag para evitar leakage:

Feature	Definición	¿Usa lag?
densidad_estudiantes_m2	Matrícula institución-región / M ² construidos	No
matriculas_sobre_pib	Matrícula regional / PIB regional	No
ratio_docente_estudiante	N° docentes / matrícula institución-región (clip p99)	No
densidad_competitiva	N° instituciones que ofrecen la misma carrera en la región-año	No
arancel_relativo_regional	Arancel / mediana regional de aranceles	No
selectividad_relativa	Puntaje de corte / media institucional de puntajes	No
concentracion_matricula_lag	Matrícula_lag de la fila / matrícula_lag total de la institución	Sí
pct_matricula_regional_lag	Matrícula_lag de la fila / matrícula_lag de la carrera en la región	Sí

9.2 ★ Anti-leakage: por qué cuatro features usan lag

Esta es una decisión anti-leakage crítica. Las features de concentración y participación de mercado podrían calcularse con la matrícula del año actual. **Hacerlo sería leakage:** estaríamos usando información del presente (la matrícula que el modelo intenta predecir) para construir un predictor.

La solución es calcular estas features con la matrícula del **año anterior** (`matriculas_lag1`), que es información legítimamente disponible al momento de predecir:

```
df_clean['_mat_lag_inst'] = df_clean.groupby(
    ['NOMBRE INSTITUCIÓN', 'AÑO_AUX'])['matriculas_lag1'].transform('sum')
```

```
df_clean['concentracion_matricula_lag'] = np.where(
    (df_clean['_mat_lag_inst'] > 0) & (df_clean['matriculas_lag1'].notna()),
    df_clean['matriculas_lag1'] / df_clean['_mat_lag_inst'], np.nan)
```

El sufijo `_lag` en el nombre de estas features es deliberado: documenta en el propio nombre que la feature respeta la restricción temporal.

9.3 Ratios docentes en lugar de conteos crudos

Una decisión clave de ingeniería: el cuerpo docente se representa mediante **ratios de composición** (`pct_doctores`, `pct_magister`, `pct_especialistas`, `pct_horas_jornada_completa`), no mediante conteos absolutos. La razón es evitar **multicolinealidad severa**: los conteos crudos de docentes por grado están altamente correlacionados entre sí y con el tamaño de la institución, produciendo factores de inflación de varianza (VIF) extremos. Los ratios normalizan por tamaño y capturan la composición del claustro, que es la señal relevante.

PARTE IV — MODELADO

10. Estrategia de validación temporal

10.1 Por qué NO se usa partición aleatoria

La decisión metodológica más importante del modelado: **no se usa train/test split aleatorio**. Un split aleatorio mezclaría años en train y test, permitiendo que el modelo "vea el futuro" durante el entrenamiento. En un problema de forecasting, esto produce métricas optimistas que no se sostienen en producción, porque en producción uno siempre predice un año que no ha ocurrido.

10.2 Arquitectura de validación de tres niveles

Nivel 1 — Holdout final (2025). El año 2025 se reserva completamente. No participa en CV, ni en tuning, ni en selección de modelo. Es la prueba ciega final.

Nivel 2 — Expanding window CV (sobre 2019-2024). El cross-validation usa una ventana creciente. Cada fold entrena con todos los años hasta $t - 1$ y valida en t :

Fold 1: train=[2018]	→ val=2019	(n_train=9.655)
Fold 2: train=[2018, 2019]	→ val=2020	(n_train=19.370)
Fold 3: train=[2018, 2019, 2020]	→ val=2021	(n_train=28.902)
Fold 4: train=[2018, 2019, 2020, 2021]	→ val=2022	(n_train=38.305)

```
Fold 5: train=[2018,2019,2020,2021,2022]      → val=2023  (n_train=47.728)
Fold 6: train=[2018,2019,2020,2021,2022,2023] → val=2024  (n_train=57.037)
```

Este esquema replica exactamente la situación de producción: predecir un año usando solo el pasado. Es más conservador (y más honesto) que un k-fold aleatorio.

Nivel 3 — Predicciones OOF retrospectivas. Para el análisis final, cada año 2019-2025 se predice con un modelo entrenado solo en años previos. Esto genera predicciones temporalmente honestas para todo el histórico.

10.3 Consecuencia: el tamaño de entrenamiento crece con el año

Una implicación directa de este esquema es que predecir 2019 usa ~9.700 filas (solo 2018), mientras que predecir 2025 usa ~66.000 filas. Esto explica por qué el desempeño del modelo mejora en años recientes: no es azar, es que dispone de más información. Este patrón se documenta cuantitativamente en §16.

11. ★ Tratamiento empírico del shock pandémico

11.1 El problema de las dummies arbitrarias

Un revisor crítico atacaría inmediatamente cualquier dummy de COVID definida a mano ("¿por qué 2020 y no 2021? ¿por qué estos años?"). Para blindar esta decisión, el pipeline **detecta los años de shock automáticamente** mediante dos criterios estadísticos, sin intervención manual.

11.2 Criterio 1: Z-score sobre la distribución del target

Se calcula la media y desviación estándar del target para cada año, y se establece un baseline con los años "normales" (2018, 2019, 2023, 2024, 2025). Luego se mide cuántas desviaciones estándar se aparta cada año candidato (2020, 2021, 2022):

```
# Baseline de años normales
μ_means, σ_means = means_normales.mean(), means_normales.std()
μ_stds,  σ_stds  = stds_normales.mean(),  stds_normales.std()

# Z-score de cada año candidato
Z_mean = (mean_target_año - μ_means) / σ_means
Z_std  = (std_target_año  - μ_stds)  / σ_stds

# Umbral 95%
```

```
if abs(Z_mean) > 1.96: → shock_nivel
if abs(Z_std) > 1.96: → shock_dispersion
```

El resultado empírico sobre el dataset v3:

Año	Z(mean)	Z(std)	Clasificación
2020	-3.39	-0.96	shock_nivel
2021	-0.14	+1.24	no detectado
2022	-3.18	+0.61	shock_nivel

Los años 2020 y 2022 superan ampliamente el umbral de -1.96 en la media del target (caída estructural), mientras que 2021 no supera ningún umbral. La detección es puramente data-driven.

11.3 Criterio 2: variación agregada como red de seguridad

Como segundo criterio, se examina la variación interanual del total agregado de matrículas. Cualquier año con variación negativa no capturado por el Criterio 1 se clasificaría como "shock de recaída". En el dataset v3, 2022 tiene variación agregada de -0.64%, pero ya fue capturado por el Criterio 1, de modo que no quedan años para esta categoría:

```
shock_nivel_years: [2020, 2022]
shock_dispersion_years: []
shock_recaida_years: []
```

11.4 Tratamiento: dummies sin downweight

Detectados los años, se agregan como dummies binarias (`es_shock_nivel` , etc.). Una decisión adicional: **no se reduce el peso (sample weight) de las observaciones pandémicas**. La razón es empírica: el análisis de varianza no mostró que los años de shock tuvieran varianza inflada que justificara reducir su influencia. La varianza del target en 2020-2022 es comparable a la de años normales (la diferencia está en la media, no en la dispersión). Downponderar habría descartado información útil sin justificación.

```
# Sample weights desactivados (peso uniforme)
# El tratamiento opera vía dummies es_shock_*, no vía downweight
covid_weight = 1.0
```

11.5 Por qué la simplificación a "solo shock de nivel" es un hallazgo, no una pérdida

Con datasets más pequeños, este mismo método podría clasificar los tres años pandémicos como tipos distintos de shock. El hecho de que con 75.622 observaciones converja a una caracterización parsimoniosa (solo dos años, solo shock de nivel) indica que el método es **robusto a la cantidad de datos**: identifica consistentemente los años verdaderamente anómalos y no sobreajusta categorías. El COVID afectó la matrícula universitaria chilena fundamentalmente como una caída sostenida del nivel, no como un aumento de la varianza interna.

12. ★ Target encoding fold-aware y el experimento de leakage

12.1 El problema de las categóricas de alta cardinalidad

Cuatro variables tienen cardinalidad demasiado alta para one-hot encoding: `NOMBRE_CARRERA` (1.355 categorías), `NOMBRE_INSTITUCIÓN` (62), `REGIÓN` (16), `area_conocimiento` (11). Hacer OHE de `NOMBRE_CARRERA` produciría 1.355 columnas dispersas. La alternativa es **target encoding**: reemplazar cada categoría por la media del target condicional a esa categoría, con suavizado bayesiano.

12.2 La trampa del leakage

El error común —y el que este pipeline evita y luego cuantifica— es **precalcular el target encoding sobre todo el dataset antes de partir en train/test**. Esto filtra información de las etiquetas de validación hacia el entrenamiento, porque la media de cada categoría incluye filas que luego estarán en validación.

La implementación correcta es **fold-aware**: el TE se recalcula dentro de cada fold del CV, usando solo los datos de entrenamiento de ese fold. El pipeline lo logra encapsulando el `TargetEncoder` de `category_encoders` dentro del `Pipeline` de scikit-learn:

```
from category_encoders import TargetEncoder

pipeline = Pipeline([
    ("pre", ColumnTransformer([
        ("te", TargetEncoder(smoothing=30), CAT_FOR_TE),    # se reajusta por
fold
        ("num", "passthrough", num_features),
    ])),
    ("model", modelo),
])
```

Al estar dentro del Pipeline, scikit-learn garantiza que `fit` del encoder se llame solo con los datos de entrenamiento de cada fold durante el CV.

12.3 El experimento controlado: cuantificación del sesgo

Para demostrar empíricamente la magnitud del leakage, el pipeline ejecuta un experimento controlado (Fase 8.2 del notebook de modelado): compara el RMSE en CV usando las columnas `te_*` precalculadas (leaky) versus el TE fold-aware. Todo lo demás se mantiene idéntico.

Configuración	RMSE CV
TE-leaky (precalculado sobre todo el dataset)	0.3005
TE-fold-aware (correcto)	0.3111
Diferencia	-0.0106
Sesgo de optimismo	3.4%

El TE leaky reporta un RMSE 3.4% mejor de lo que el modelo realmente logra. Esta magnitud (superior al 1-2% típico de la literatura) se debe a la altísima cardinalidad de `NOMBRE_CARRERA` : con 1.355 categorías, muchas tienen pocas observaciones, y precalcular su media filtra señal del futuro de forma pronunciada.

12.4 Por qué esto importa para la defensa

Este experimento es una de las defensas más fuertes del documento. Demuestra que (i) el equipo conoce el riesgo de leakage, (ii) lo evita correctamente, y (iii) cuantifica honestamente cuánto habría inflado las métricas el atajo incorrecto. Un revisor no puede acusar al modelo de leakage por TE: el pipeline ya lo midió y lo evitó.

13. Selección de modelos

13.1 ★ Por qué gradient boosting (y no deep learning ni modelos lineales)

La elección de la familia de modelos se justifica por la naturaleza del problema:

Por qué no deep learning: la literatura es consistente en que para datos tabulares de cardinalidad moderada, los métodos de gradient boosting igualan o superan a las redes neuronales, con menor costo computacional y mayor interpretabilidad [Shwartz-Ziv & Armon,

2022]. El dataset tiene 75.622 filas y ~49 features: un régimen donde las redes profundas no aportan ventaja y sí agregan complejidad, riesgo de sobreajuste y opacidad.

Por qué no modelos lineales como modelo final: la relación entre las features y la tasa de crecimiento es no-lineal y con interacciones (el efecto del arancel depende de la región, el efecto del puntaje depende del área, etc.). Un modelo lineal no captura estas interacciones sin ingeniería manual exhaustiva. No obstante, sí se usa un modelo lineal (Ridge) como baseline informado (§13.2), para cuantificar cuánto aporta la no-linealidad.

Por qué gradient boosting específicamente: maneja nativamente valores faltantes, es robusto a multicolinealidad, captura no-linealidades e interacciones automáticamente, y ofrece interpretabilidad vía SHAP. Es el estado del arte para este tipo de problema.

13.2 Baselines informados

Para contextualizar el desempeño del modelo, se evalúan cuatro baselines de complejidad creciente. Sin baselines, un R^2 o un WAPE carecen de referencia: no se sabe si son buenos o malos para este problema.

Baseline	Descripción	test_RMSE	test_R ²
B0	Mediana constante del train	0.3214	-0.0022
B1	Media histórica por (institución, carrera)	0.3263	-0.0331
B2	Ridge solo con <code>matriculas_lag1</code>	0.3223	-0.0079
B3	Ridge con feature set completo	0.3103	0.0655

El salto de B2 (0.3223, solo lag) a B3 (0.3103, features completas) cuantifica el aporte de las features contextuales en un marco lineal. El modelo final (CatBoost, RMSE 0.2837 en holdout escala arcsinh) supera ampliamente a todos, cuantificando el aporte de la no-linealidad.

13.3 Por qué CatBoost (y honestidad sobre el margen)

Se evaluaron tres implementaciones de gradient boosting con tuning de Optuna. El resultado:

Modelo (tuneado)	RMSE CV
CatBoost	0.3111
LightGBM	0.3116
XGBoost	0.3125

CatBoost gana, pero **el margen es honestamente marginal** (0.0005 sobre LightGBM). El documento no exagera este resultado: para este problema, la familia de gradient boosting es robusta a la elección específica de implementación. CatBoost se selecciona por su victoria consistente (también ganó en la versión anterior del dataset) y por su manejo nativo de categóricas con *ordered boosting*, que reduce el target leakage interno. Pero se reconoce explícitamente que LightGBM o XGBoost habrían producido resultados prácticamente equivalentes.

Esta honestidad es deliberada: afirmar que CatBoost es "claramente superior" con una diferencia de 0.0005 sería estadísticamente insostenible y un revisor lo detectaría.

13.4 ★ El experimento de clustering que se descartó

Una pregunta natural de un revisor: "¿probaron agrupar instituciones similares (clustering) como feature adicional?". La respuesta es sí, y se descartó con evidencia.

El experimento (Fase 6.5 del notebook): se construyó un perfil estructural de cada institución (composición docente, aranceles, selectividad, densidad), se aplicó K-Means, y se determinó el K óptimo por silhouette score (K=3). Luego se midió el uplift de agregar el `cluster_id` como feature en el modelo:

```
K_OPT = 3    # óptimo por silhouette
# ... asignación de clusters ...

# Test de uplift: LightGBM con vs sin cluster
CV RMSE sin cluster: 0.3144
CV RMSE con cluster: 0.3146
Uplift (positivo = mejora): -0.0002

INCLUIR_CLUSTER = uplift > 0.001    # → False
Decisión: NO incluir cluster_arquetipo en modelos finales
```

El uplift fue **-0.0002** (es decir, agregar el cluster empeoró marginalmente el modelo). La interpretación: la información estructural que el clustering capturaría ya está disponible para el modelo de forma más granular a través del target encoding de `NOMBRE INSTITUCIÓN` y las features de claustro. El cluster es una compresión con pérdida de información que el modelo ya tiene en mejor forma.

Por qué documentar un experimento fallido: esto es lo que distingue una metodología rigurosa. No se dice "no usamos clustering" (lo que invitaría a la pregunta "¿por qué no?"). Se dice "lo probamos, medimos el uplift, fue negativo, lo descartamos". Un revisor no puede atacar una decisión respaldada por evidencia empírica.

14. Optimización de hiperparámetros

14.1 Optuna con TPE y pruning

Los tres modelos de gradient boosting se optimizan con Optuna usando un *Tree-structured Parzen Estimator* (TPE) como sampler, que modela la distribución de hiperparámetros prometedores de forma más eficiente que grid o random search. Se complementa con `MedianPruner` para abortar tempranamente los trials poco prometedores:

```
sampler = optuna.samplers.TPESampler(seed=SEED)
pruner  = optuna.pruners.MedianPruner(n_startup_trials=10, n_warmup_steps=2)
study   = optuna.create_study(direction="minimize", sampler=sampler,
pruner=pruner)
study.optimize(objective, n_trials=50)
```

14.2 Función objetivo: RMSE promedio en CV temporal

Cada trial se evalúa sobre los seis folds del expanding window CV, y la métrica objetivo es el RMSE promedio en escala arcsinh. Esto garantiza que el tuning optimice para generalización temporal, no para un split arbitrario.

14.3 Número de trials

Se ejecutan 50 trials por modelo. Esta cantidad balancea exploración del espacio de hiperparámetros con costo computacional. Con TPE + pruning, 50 trials son suficientes para converger a una región de buen desempeño; trials adicionales mostraban rendimientos marginales decrecientes.

(Continúa en la Parte V — Resultados y su interpretación)

PARTE V — RESULTADOS Y SU INTERPRETACIÓN

15. Métricas: cuáles, por qué esas, y qué NO dicen

15.1 ★ La distinción tasa vs nivel (la defensa del R^2)

Esta es la sutileza metodológica más importante para interpretar los resultados, y la que un revisor atacaría primero.

El modelo se optimiza sobre la **tasa de crecimiento** (escala arcsinh). En esa escala, el R^2 en holdout es ~ 0.22 . Un revisor desprevenido vería ese número y concluiría que el modelo es malo.

Pero la tasa de crecimiento es, por naturaleza, casi-ruido: predecir si una carrera crecerá 3% o 5% el próximo año es intrínsecamente difícil porque depende de factores idiosincráticos no observables. Lo que importa para planificación no es la tasa, sino el **número de matrículas resultante**.

Cuando se reconstruye la matrícula absoluta ($\text{lag} \times (1 + \text{tasa predicha})$) y se mide el R^2 sobre esa magnitud, el resultado es **0.936**. La diferencia se explica porque el lag aporta la mayor parte de la señal de nivel, y el modelo solo necesita predecir el ajuste marginal.

Ambas métricas son correctas y se reportan ambas. La de tasa (0.22) describe la dificultad del problema marginal; la de nivel (0.936) describe la utilidad práctica del modelo. Reportar solo la primera subestimaría groseramente el valor del modelo; reportar solo la segunda podría parecer inflado sin la explicación. El documento reporta ambas con su interpretación, que es la práctica honesta.

15.2 Por qué WAPE y no MAPE

El error porcentual se mide con **WAPE** (Weighted Absolute Percentage Error), no con MAPE (Mean Absolute Percentage Error). La razón es técnica:

$$\text{MAPE} = \frac{100}{n} \sum \frac{|y_i - \hat{y}_i|}{y_i} \quad \text{WAPE} = 100 \cdot \frac{\sum |y_i - \hat{y}_i|}{\sum y_i}$$

El MAPE divide cada error por su valor real individual, lo que lo hace **explotar con valores pequeños**: una carrera con 10 alumnos donde el modelo predice 13 tiene 30% de error, que pesa lo mismo que un error del 30% en una carrera de 800. El MAPE sobre-pondera los programas minúsculos.

El WAPE pondera por el volumen real: el error total absoluto sobre el total real. Es la métrica correcta para datos con rangos heterogéneos y permite la interpretación directa "el modelo acierta el $(100 - \text{WAPE})\%$ del total agregado".

15.3 ★ Skill score: la métrica que un revisor respeta

La métrica más persuasiva es el **skill score** sobre el baseline naïve. El baseline naïve predice que la matrícula del año t es igual a la del año $t - 1$ (persistencia). Es el predictor trivial contra el que todo modelo de forecasting debe demostrar valor.

$$\text{Skill} = 1 - \frac{\text{RMSE}_{\text{modelo}}^2}{\text{RMSE}_{\text{naïve}}^2}$$

El resultado en holdout 2025: **+77.4%**. Esto significa que el modelo reduce el error cuadrático en 77.4% respecto a simplemente repetir el año anterior. En la literatura de forecasting, skill scores de +10% a +30% ya se consideran buenos. Un +77% es excepcional y constituye la evidencia más fuerte de que el modelo captura dinámicas reales, no solo inercia.

15.4 Las ocho métricas reportadas

El documento reporta ocho métricas complementarias en lugar de una sola, porque cada una ilumina un aspecto distinto y ninguna es suficiente por sí sola:

Métrica	Qué captura	Limitación que tiene sola
RMSE	Error promedio, penaliza grandes	Sensible a outliers y escala
MAE	Error promedio robusto	No distingue grandes de pequeños
MedAE	Error mediano, muy robusto	Ignora la cola
R²	Varianza explicada	Sensible a la varianza del target
WAPE	Error porcentual ponderado	No muestra dispersión
MAPE	Error porcentual promedio	Explota con valores chicos
sMAPE	Error porcentual simétrico	Menos interpretable
Skill score	Mejora sobre naïve	Requiere baseline

Reportar las ocho previene la crítica de "elegiste la métrica que te conviene".

16. Análisis macro → micro

16.1 El hallazgo central: la precisión crece con la agregación

El desempeño del modelo no es un número único: depende del nivel de agregación al que se consulta. El análisis mide las métricas en ocho niveles, desde la granularidad fina hasta el agregado nacional anual.

Holdout 2025:

Nivel	n grupos	WAPE (%)	Confianza (%)
1. Nacional anual	1	0.02	99.98
2. + Institución	55	2.79	97.21
3. + Área conocimiento	418	5.20	94.80

Nivel	n grupos	WAPE (%)	Confianza (%)
4. + Región	688	6.09	93.91
5. + Carrera	2.770	10.09	89.91
6. + Condición	4.829	12.44	87.56
7. + Género	8.446	14.38	85.62
8. + Jornada (fina)	9.072	15.05	84.95

16.2 Por qué esto es estadísticamente esperado (no un truco)

Un revisor podría sospechar que la confianza del 99.98% a nivel nacional es un artefacto. No lo es: es una consecuencia matemática de la **cancelación de errores idiosincráticos**. Cuando se agregan miles de predicciones, los errores positivos y negativos se compensan parcialmente. Si el modelo no tiene sesgo sistemático (y el análisis de residuos confirma que su sesgo es cercano a cero), los errores a nivel fino se cancelan al agregar, dejando un error agregado pequeño.

Esto no significa que el modelo sea "perfecto" a nivel nacional: significa que es **insesgado**. La precisión del 99.98% refleja ausencia de sesgo sistemático, lo cual es exactamente lo que se requiere para planificación agregada.

16.3 La implicación práctica honesta

El documento es explícito sobre qué uso es apropiado: el modelo es **excelente para planificación agregada** (nacional, regional, institucional: confianza 94-99.98%) y **útil con cautela para predicción de programa específico** (granularidad fina: 85%). No se afirma que el modelo prediga perfectamente cada programa: se afirma que es altamente confiable para los niveles agregados que importan para política pública y planificación institucional.

17. Forecast 2026 y su validación

17.1 El pronóstico

El modelo proyecta **707.960 estudiantes universitarios** para 2026, un crecimiento de **+0.16%** sobre 2025.

17.2 Validación contra baselines

Un forecast sin validación es una apuesta. El documento valida el pronóstico contra tres baselines:

Método	Crecimiento 2026	Diferencia vs modelo
Modelo CatBoost	+0.16%	—
Regresión lineal 2018-2025	-1.65%	+1.81 pp
Promedio variaciones 2023-2025	+2.41%	-2.25 pp
Variación 2025	+2.15%	-2.00 pp

17.3 Por qué el +0.16% es defendible (no sospechosamente bajo)

Un revisor podría objetar: "los últimos años crecieron 2-3%, ¿por qué tu modelo predice apenas 0.16%?". La respuesta tiene dos componentes:

Componente 1 — Desaceleración del rebote post-pandémico. Las dummies de shock (2020, 2022) le indican al modelo que esos años fueron anómalos. El modelo aprende que el crecimiento de 2023-2024 fue en parte un rebote de recuperación tras las caídas pandémicas, no una tendencia de crecimiento sostenido. Extrapolar ese rebote linealmente sobrestimaría 2026.

Componente 2 — Contexto estructural de pérdida de participación universitaria. El modelo cubre solo universidades. Los datos del propio SIES muestran que la participación universitaria en el sistema de educación superior cae sostenidamente: del 57.1% (2018) al 55.2% (2025). El subsector técnico-profesional (IP/CFT) crece más rápido. En este contexto, un crecimiento universitario modesto es coherente: la demanda estudiantil está migrando parcialmente hacia el subsector no cubierto por el modelo.

El forecast no es sospechoso: es la lectura correcta de un subsistema que crece lentamente mientras pierde participación relativa.

18. Explicabilidad SHAP

18.1 Las features más influyentes

El análisis SHAP global identifica las contribuciones de cada feature:

#	Feature	SHAP medio absoluto
1	CONDICION_NUEVO	0.0574
2	NOMBRE CARRERA	0.0558
3	matriculas_lag1	0.0548

#	Feature	SHAP medio absoluto
4	pct_matricula_regional_lag	0.0389
5	NOMBRE INSTITUCIÓN	0.0240

18.2 El hallazgo interpretable

CONDICION_NUEVO es la feature más importante. Esto refleja una realidad estructural: la dinámica de alumnos nuevos (gobernada por admisión y captación) es fundamentalmente distinta de la de alumnos antiguos (gobernada por retención). El modelo aprende a tratar estos dos segmentos de forma diferenciada, lo cual es coherente con cómo funciona realmente el sistema.

18.3 Coherencia del tratamiento pandémico

Las dummies `es_shock_*` no aparecen en el top 15 de SHAP. Esto indica que CatBoost captura los efectos temporales principalmente vía `AÑO_AUX` (que sí aparece). Las dummies cumplen un rol de robustez metodológica (hacer el tratamiento explícito y trazable) más que de aporte predictivo directo. Esto es consistente y se documenta honestamente: el tratamiento no introduce ruido y aporta trazabilidad, aunque su contribución predictiva marginal sea baja en un modelo de árboles.

PARTE VI — BLINDAJE

19. ★★ Anticipación de objeciones (Q&A adversarial)

Esta sección responde, con evidencia empírica del pipeline, las objeciones que un ingeniero o científico de datos crítico plantearía. Cada respuesta se ancla en datos concretos, no en apelaciones a autoridad.

P1. "Tu R^2 de 0.936 está inflado. Mediste sobre el nivel de matrículas, donde el lag hace casi todo el trabajo. El R^2 real sobre la tasa es 0.22."

Ambos números se reportan explícitamente y miden cosas distintas. El R^2 de 0.22 sobre la tasa describe la dificultad del problema marginal (predecir el crecimiento incremental, que es casi-ruido). El R^2 de 0.936 sobre el nivel describe la utilidad práctica para planificación. No hay inflado: hay dos métricas para dos preguntas. La prueba de que el modelo aporta valor real más allá del lag es el **skill score de +77.4% sobre el baseline naïve** (que es exactamente

"usar solo el lag"). Si el lag hiciera todo el trabajo, el skill score sería cercano a cero. Un +77% demuestra que el modelo aporta sustancialmente sobre la persistencia.

P2. "El target encoding sobre NOMBRE CARRERA con 1.355 categorías es leakage puro."

Sería leakage si se precalculara sobre todo el dataset. No es el caso: el TE se implementa fold-aware dentro del Pipeline de scikit-learn, reajustándose en cada fold solo con datos de entrenamiento. Además, el pipeline **cuantifica explícitamente** cuánto leakage habría producido el atajo incorrecto: un experimento controlado (§12.3) muestra que el TE precalculado infla el RMSE en 3.4%. El equipo no solo evita el leakage: lo mide para demostrar que lo entiende y lo controla.

P3. "Usaste validación temporal, pero igual podrías tener leakage por las features construidas con datos del año actual."

Las features de concentración y participación de mercado se construyen explícitamente con el **lag** (matrícula del año anterior), no con la matrícula actual. El sufijo `_lag` en sus nombres (`concentracion_matricula_lag`, `pct_matriculas_regional_lag`) lo documenta. El código (§9.2) muestra que se calculan con `matriculas_lag1`, información legítimamente disponible al momento de predecir. No hay features que usen el target ni la matrícula del año que se predice.

P4. "Perdiste el 64% de las filas del SIES filtrado (114K → 75K). ¿Qué estás escondiendo?"

La caída se descompone en filtros auditables (§5.2): (1) filas sin lag calculable (primera aparición de combinaciones), y (2) filtro `matriculas_lag1 ≥ 10` que elimina combinaciones de cola larga con menos de 10 estudiantes, porque generan tasas numéricamente inestables. Crucialmente, aunque se elimina el 31% de las *filas* en el último stage, se conserva el **97-99% de las matrículas** cada año (§5.1). Esto prueba que lo eliminado son programas minúsculos, no masivos. La señal agregada se preserva casi íntegra. Todo el procedimiento es reproducible desde el SIES público (§5.3).

P5. "Las dummies de COVID son arbitrarias. Elegiste 2020 y 2022 a conveniencia."

No se eligieron a mano: se detectaron automáticamente por Z-score con umbral 95% (§11.2). Los datos muestran $Z(\text{mean})$ de -3.39 para 2020 y -3.18 para 2022, ambos superando ampliamente el umbral de -1.96, mientras que 2021 tiene $Z(\text{mean})$ de -0.14 (no significativo). La selección es puramente data-driven y reproducible. El código de detección está en el documento.

P6. "Tu modelo no le gana a un baseline trivial."

Le gana a cuatro baselines, incluido el naïve de persistencia. El skill score es +77.4%. En escala arcsinh, el RMSE del modelo (0.2837) supera a todos los baselines (B0: 0.3214, B1: 0.3263, B2: 0.3223, B3: 0.3103). El salto de B3 (mejor baseline, Ridge full: 0.3103) al modelo (0.2837) cuantifica el aporte de la no-linealidad capturada por gradient boosting.

P7. "El forecast de +0.16% es sospechosamente bajo. Los últimos años crecieron más."

Es defendible por dos razones (§17.3): (1) el modelo aprende que el crecimiento 2023-2024 fue parcialmente un rebote post-pandémico que se desacelera, gracias a las dummies de shock; (2) el contexto estructural muestra que la participación universitaria cae del 57.1% al 55.2% en el período, porque IP/CFT crecen más rápido. El modelo cubre solo universidades, y un crecimiento modesto es coherente con un subsistema que pierde participación relativa. Además, el forecast se valida contra tres baselines y queda en un rango razonable entre ellos.

P8. "Imputaste el 73% de las filas. Tus features son mayormente ruido inventado."

La imputación es jerárquica e informativa, no un relleno global (§8). El 69.8% del trabajo de imputación se resuelve en la Capa 2, que usa la mediana por área de conocimiento en la institución y año: un programa de ingeniería sin dato hereda el comportamiento típico de ingeniería en su institución, no un valor global. Solo el 26% cae a mediana global. Además, cada fila lleva la flag `fue_imputado` que el modelo usa como feature, permitiéndole ponderar la incertidumbre de los datos imputados. La imputación es trazable y se le da al modelo la capacidad de ajustar por ella.

P9. "CatBoost ganó por azar. La diferencia con LightGBM es 0.0005, que es ruido."

El documento lo reconoce explícitamente (§13.3): el margen es marginal y la familia de gradient boosting es robusta a la elección de implementación. CatBoost se elige por victoria

consistente (también ganó en la versión previa del dataset) y por su manejo nativo de categóricas, pero se afirma claramente que LightGBM o XGBoost producirían resultados equivalentes. No se exagera la superioridad de CatBoost.

P10. "¿Por qué no probaste clustering de instituciones? Podría mejorar el modelo."

Se probó y se descartó con evidencia (§13.4). Se construyó un perfil estructural por institución, se aplicó K-Means (K=3 óptimo por silhouette), y se midió el uplift de agregar el cluster como feature: fue **-0.0002** (empeoró marginalmente). La información que el clustering capturaría ya está disponible para el modelo de forma más granular vía el target encoding de la institución y las features de claustro. El experimento está documentado con su código y resultado.

P11. "WAPE esconde la dispersión de los errores. Un WAPE de 15% puede ocultar errores enormes en programas específicos."

Por eso se reportan ocho métricas, no solo WAPE (§15.4). El RMSE captura la penalización de errores grandes, el MedAE la robustez, y el análisis de errores extremos (top 15 peores predicciones) identifica explícitamente dónde el modelo falla más. El documento no oculta la dispersión: la reporta a través de múltiples métricas y del análisis macro→micro que muestra cómo el error varía con la granularidad.

P12. "No reportas intervalos de confianza ni incertidumbre en las predicciones."

Correcto, y es una limitación reconocida (§20). El modelo actual produce predicciones puntuales. La cuantificación de incertidumbre (vía quantile regression o conformal prediction) es trabajo futuro identificado. No obstante, el análisis macro→micro y el backtesting retrospectivo proveen una caracterización empírica del error esperado a cada nivel de agregación, que funciona como una banda de confianza implícita validada históricamente.

P13. "Tu CV temporal tiene folds con muy pocos datos al inicio (Fold 1 solo tiene 2018). Eso sesga la evaluación."

Es una característica deliberada del expanding window, no un defecto (§10.2). Replica exactamente la situación de producción: al predecir 2019, solo se dispone de 2018. Reportar el desempeño de esos folds tempranos es honesto, no sesgado. El análisis por año (§16)

muestra explícitamente cómo el desempeño mejora con más datos, lo cual es información valiosa sobre la madurez del modelo, no un artefacto a ocultar.

P14. "El modelo solo cubre universidades. Tu forecast no sirve para el sistema de educación superior."

Correcto y explícito desde el §1.2. El alcance es deliberadamente universitario. El documento no afirma cubrir IP/CFT; al contrario, usa la dinámica de pérdida de participación universitaria para contextualizar el forecast. La extensión a IP/CFT es la dirección futura prioritaria identificada. El modelo responde correctamente la pregunta que se propone responder: matrícula universitaria.

P15. "¿Cómo sé que no sobreajustaste los hiperparámetros al holdout 2025?"

El holdout 2025 nunca participó en el tuning (§10.2). Optuna optimizó sobre el expanding window CV de 2019-2024 exclusivamente. El holdout se evaluó una sola vez, al final, con el modelo ya seleccionado y tuneado. No hubo iteración sobre el holdout. Además, el desempeño en OOF retrospectivo (R^2 0.887 sobre todo 2019-2025) es consistente con el holdout (0.936), lo que descarta sobreajuste a un año específico.

P16. "La winsorización al 1-99% es arbitraria y podría estar ocultando casos reales."

La winsorización limita outliers extremos que corresponden a cambios administrativos (aperturas/cierres de programas), no a dinámicas de demanda (§3.4). Se aplica solo a los percentiles extremos (2% de los datos) y no elimina filas, solo acota valores. Sin ella, una carrera que pasa de 2 a 40 alumnos (+1900%) dominaría el entrenamiento con ruido. Es una práctica estándar para estabilizar el target, aplicada de forma transparente y simétrica.

P17. "Los ratios docentes podrían tener multicolinealidad entre sí."

Por eso se usan ratios de composición en lugar de conteos crudos (§9.3). Los conteos absolutos de docentes por grado tienen VIF extremos (correlacionados entre sí y con el tamaño institucional). Los ratios (`pct_doctores` , etc.) normalizan por tamaño y reducen drásticamente la multicolinealidad. Además, los modelos de gradient boosting son inherentemente robustos a multicolinealidad (a diferencia de los lineales), por lo que el riesgo es bajo en el modelo final.

P18. "¿Por qué arcsinh y no log o Box-Cox?"

Porque la tasa de crecimiento es frecuentemente negativa (§3.2), y el logaritmo no admite valores ≤ 0 . Box-Cox tampoco maneja negativos sin desplazamiento arbitrario. El arcsinh está definido en todo $\mathbb{R}$, es simétrico, preserva el signo, se comporta como identidad cerca de cero y como log en las colas. Es la transformación natural para variables con signo y colas pesadas, sin parámetros arbitrarios que ajustar.

P19. "Tu cobertura del 96% podría ser selectiva: descartaste lo que el modelo predice mal."

No hay selección por desempeño. Los criterios de exclusión son puramente estructurales y se aplican *antes* de cualquier modelado (§5.2): filas sin lag (imposibilidad matemática de calcular target) y filtro de cola larga ($\text{lag} \geq 10$, por inestabilidad numérica). Ninguna fila se excluye por su error de predicción. La reconciliación año a año (§5.4) muestra que la cobertura del 94.8-98.0% es uniforme, no concentrada en casos "fáciles".

P20. "Las features macroeconómicas (PIB, desempleo) tienen baja correlación con el target. ¿Por qué incluirlas?"

Las correlaciones lineales bivariadas con el target son efectivamente bajas (`matriculas_sobre_pib`: -0.011). Pero esto mide solo relación lineal directa; los modelos de gradient boosting capturan interacciones no-lineales (el efecto del PIB puede depender de la región y el área simultáneamente). Su inclusión se justifica empíricamente: el modelo completo supera al baseline que no las usa. Además, su baja importancia SHAP (no están en el top 15) es información honesta: aportan poco, pero no dañan, y completan el contexto macroeconómico que un revisor de dominio esperaría ver considerado.

20. Limitaciones reconocidas

El documento reconoce explícitamente las siguientes limitaciones, porque ocultarlas debilitaría la defensa:

1. **Cobertura limitada a universidades.** Excluye IP/CFT (~45% del sistema). Extensión prioritaria pendiente.

2. **Predicciones puntuales sin intervalos de confianza.** Quantile regression / conformal prediction es trabajo futuro.
 3. **Desempeño menor en regiones pequeñas.** Aysén, Atacama y Magallanes tienen menos datos y mayor error. Modelado jerárquico bayesiano podría ayudar.
 4. **Forecast bajo supuesto de estabilidad macroeconómica.** Asume PIB y desempleo 2026 $\approx$ 2025. Sin análisis de sensibilidad a escenarios.
 5. **Naturaleza asociativa, no causal.** SHAP identifica asociaciones, no causas. No permite afirmaciones contrafactuales.
 6. **Dependencia de la calidad de imputación.** El 26% de imputación terminal (mediana global) provee señal débil en esos casos, mitigada por la flag `fue_imputado`.
 7. **El target marginal es difícil.** El R^2 de 0.22 sobre la tasa refleja que el crecimiento incremental tiene un componente irreducible de ruido idiosincrático.
-

21. Reproducibilidad

21.1 Control de aleatoriedad

Todas las fuentes de aleatoriedad usan una semilla fija (`SEED`), aplicada a: KMeans, los tres modelos de gradient boosting, el TPESampler de Optuna, y los muestreos de SHAP. Esto garantiza que la ejecución sea determinística.

21.2 Versiones de librerías

El pipeline fue validado con: numpy 1.26.4, pandas 2.2.0, scipy 1.13.1, scikit-learn 1.5.0, lightgbm 4.5.0, xgboost 2.1.1, catboost 1.2.5, optuna 4.0.0, shap 0.45.1, category_encoders 3.0.1.

 La compatibilidad entre numpy, scipy y scikit-learn es sensible. La combinación documentada está validada; cambios mayores de versión pueden requerir ajustes.

21.3 Orden de ejecución

Los notebooks deben ejecutarse en orden estricto: (1) consolidado $\rightarrow$ produce `dataset_modelado_final.csv`; (2) modelado $\rightarrow$ produce el parquet de predicciones; (3) análisis $\rightarrow$ produce figuras y tablas. Cada notebook detecta automáticamente el output más reciente del anterior.

21.4 Trazabilidad de outputs

Cada corrida del notebook de modelado genera artefactos con timestamp:

`predicciones_modelo_final_{YYYYMMDD_HHMM}.parquet`, el modelo `.joblib`, y un

`metadata_modelo_{timestamp}.json` con la configuración completa. Esto permite reconstruir exactamente qué configuración produjo qué resultados.

APÉNDICES

Apéndice A — Diccionario de columnas del dataset final

El dataset `dataset_modelado_final.csv` (75.622×51) contiene:

Categorías originales (4) — para TE fold-aware:

`REGIÓN`, `NOMBRE INSTITUCIÓN`, `NOMBRE CARRERA`, `area_conocimiento`

Identificadores y auxiliares:

`AÑO_AUX` (año), `CÓDIGO DE INSTITUCIÓN` (excluido del modelo), `matriculas_lag1` (log1p)

Target:

`tasa_crecimiento` (arcsinh de la tasa winsorizada)

Features de precio y selectividad:

`valor_arancel` (log1p), `valor_matricula` (log1p), `puntaje_maximo`, `puntaje_minimo`, `puntaje_promedio`, `puntaje_corte_ultimo`, `duracion_semestres`, `selectividad_relativa`, `arancel_relativo_regional`

Features de claustro (ratios):

`NºDocentes` (log1p), `pct_doctores`, `pct_magister`, `pct_especialistas`, `pct_horas_jornada_completa`, `ratio_docente_estudiante`

Features de infraestructura:

`densidad_estudiantes_m2` (log1p)

Features macroeconómicas:

`pib_mm_pesos` (log1p), `tasa_desempleo`, `matriculas_sobre_pib`

Features de mercado (con lag, anti-leakage):

`densidad_competitiva`, `concentracion_matricula_lag`, `pct_matricula_regional_lag`

Features de retención:

`retencion_1er_anio`, `ingreso_4to_anio_titulacion`

Dummies one-hot:

`GENERO` (label-encoded), `JORNADA_*`, `CONDICION_NUEVO`, `CLASIFICACIÓN INSTITUCIÓN NIVEL 2_*`, `CLASIFICACIÓN INSTITUCIÓN NIVEL 3_*`, `clasificacion3_*`

Dummies de shock (tratamiento pandémico):

`es_shock_nivel` , `es_shock_dispersion` , `es_shock_recaida`

Flag de trazabilidad:

`fue_imputado`

Columnas te_* (NO usadas en el modelo, solo baseline de leakage):

`te_REGIÓN` , `te_NOMBRE INSTITUCIÓN` , `te_NOMBRE CARRERA` , `te_area_conocimiento`

Apéndice B — Configuración del modelo ganador

El modelo final es **CatBoost** tuneado con 50 trials de Optuna sobre expanding window CV. Los hiperparámetros exactos se persisten en el archivo `metadata_modelo_{timestamp}.json` generado en cada corrida (Fase 12.1 del notebook de modelado). El espacio de búsqueda de Optuna cubre profundidad de árboles, learning rate, regularización L2, número de iteraciones, y parámetros específicos de CatBoost (`border_count`, `bagging_temperature`). La configuración exacta es reproducible cargando el JSON de metadata.

Fin del documento técnico.